A3: Closing the AI-Slop Feedback Loop with Barrier-Certificate-Guided Static Analysis for Python

Anonymous Author(s)

Abstract

Large language models (LLMs) can generate candidate bug-detection heuristics, verification hypotheses, and even static analysis rules at unprecedented speed—but the vast majority of such AI-generated material is imprecise, unsound, or outright wrong. Unchecked, this creates a *negative* feedback loop—the *AI-slop feedback loop*—in which an LLM proposes analysis ideas, most are noise, that noise pollutes the next round of prompts and training data, and quality degrades with every iteration.

We present A3 (**A**dvanced **A**utomated **A**nalysis), an open-source Python static-analysis tool that *inverts* this loop, turning it from a vicious cycle into a virtuous one. A3 combines (1) bytecode-level control-flow and data-flow analysis, (2) a portfolio of 18 verification techniques drawn from the barrier-certificate, CEGAR, IC3/PDR, SOS/SDP, CHC, and invariant-synthesis literatures (the *kitchensink* engine), and (3) agentic LLM triage that investigates only the small residual set of findings that static proof and disproof both leave unresolved.

On a synthetic benchmark suite of 43 injected bugs across 67 bug types, A3 achieves 90.2% precision at 100% recall ($F_1 = 0.949$). On Microsoft DeepSpeed (~150 k LoC, 7 826 functions), the barrier-certificate engine alone proves 95.3% of 2 638 raw findings to be false positives, leaving 10 confirmed true bugs including 6 division-by-zero vulnerabilities reachable from user-supplied configuration. We further evaluate A3 against the BugsInPy benchmark of 493 real bugs across 17 open-source projects and describe an iterative self-improvement protocol in which false negatives and false positives drive automated patches to the analyser itself. Across 15 Microsoft-maintained repositories, A3 surfaces true-positive security and crash bugs in Counterfit, RD-Agent, RESTler, FLAML, and Presidio, among others.

ACM Reference Format:
Anonymous Author(s). 2026. A3: Closing the AI-Slop Feedback Loop with Barrier-Certificate-Guided Static Analysis for Python. In *Proceedings of 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASE '26, Sacramento, CA, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The last three years have produced a remarkable acceleration in AI-assisted software engineering. LLMs write code, generate tests, propose refactorings, and even suggest formal specifications. Yet a quieter, less-celebrated failure mode has emerged alongside these capabilities: *AI slop*—the unchecked proliferation of plausible-sounding but incorrect or imprecise artifacts generated by language models.

When an LLM is asked to produce static-analysis rules, verification hypotheses, or bug-detection heuristics, it generates *broadly and quickly*. The results are rarely sound. In our experience, a single prompting session can yield dozens of candidate bug-detection rules of which fewer than 20% survive contact with a real codebase. The remaining 80% produce false positives, miss edge cases, or encode assumptions that are semantically vacuous once bytecode-level control flow is considered.

Left unchecked, this process compounds into what we call the *AI-slop feedback loop*—a **vicious cycle**:

(1) AI-generated rules flood the analyser with noise.
(2) Developers lose trust and disable alerting.
(3) The feedback signal for improving the LLM's future suggestions degrades.
(4) The next generation of AI-generated rules is trained on, or prompted with, the output of the noisy system, producing *even more* noise.

This paper describes A3, a system that *flips this loop on its head*: by inserting rigorous, deterministic static-analysis machinery between the LLM's expansive generation and the developer's inbox, the same iterative cycle becomes a **virtuous cycle** in which each round of AI-generated hypotheses is pruned by formal methods, the survivors improve the analyser, and the improved analyser produces sharper feedback for the next round. The core architectural principle is **static-first, agentic-second**: deterministic, auditable, non-LLM reasoning handles the vast majority of work, and LLM judgment is reserved for the small residual where static proof and disproof both stop.

A3 is implemented as an open-source Python package (`pip install a3-python`), totalling roughly 191 k lines of Python across 67 bug-type detectors, a multi-technique verification portfolio, and a CI integration layer with baseline ratcheting.

Contributions.

- We characterise the AI-slop feedback loop—historically a degenerative vicious cycle—and show how a five-phase iterative methodology (theorise → code → test → fix code → fix theory) inverts it into a virtuous self-improvement cycle (§2).
- We present A3's *kitchensink* verification engine, which orchestrates 18 SOTA techniques from the barrier-certificate, CEGAR, IC3/PDR, SOS/SDP, CHC, ICE, Houdini, SyGuS,

and assume-guarantee literatures into a unified false-positive filter (§3).
- We describe a self-improvement protocol that uses the BugsInPy benchmark [1] of 493 real bugs across 17 open-source Python projects to iteratively patch the analyser with the help of an LLM coding assistant (§4).
- We report quantitative results on synthetic benchmarks ($F_1 = 0.949$), large-scale industrial code (DeepSpeed: 95.3% FP elimination, 10 confirmed TPs), and a portfolio of 15 Microsoft-maintained repositories with confirmed true positives (§5).

2 The AI-Slop Feedback Loop: From Vicious to Virtuous

2.1 Origin: From Quantitative Semantics to Shipping Code

The project that became A3 did not begin as a CLI tool. It began with an almost unreasonable challenge: could an LLM generate a genuinely novel verification theory at research-paper scale, then survive contact with implementation?

The earliest phase produced a 74-page manuscript on quantitative model checking. The central idea was to stop asking only "is the system safe?" and instead ask "how far is it from safety?"—treating the distance to a safety boundary as a first-class semantic object that can be computed, optimised, and used to prioritise analysis effort. Three instincts from this phase survived into the final system:

(1) Safety is not merely Boolean; uncertainty has geometric *shape*.
(2) Quantitative semantics can *prioritise* analysis work.
(3) If distance to a safety boundary is computable, *repair* can be guided.

What did not survive was the assumption that quantitative semantics alone suffices for industrial-scale bug detection. In practice, most engineering pain is not proving one deep theorem but *killing mountains of false positives without missing true bugs.*

2.2 Three Eras

The project passed through three eras, each surviving as a layer in the final architecture:

Era I: Distance to Safety. Traces modelled as distributions, properties as structured acceptance sets, distance to satisfaction as a metric. This era contributed the *confidence scoring* module.

Era II: Barrier Certificates. The system shifted from measurement to separation: construct a mathematical function $B(\mathbf{x})$ such that initial states satisfy $B(\mathbf{x}) \leq 0$, unsafe states satisfy $B(\mathbf{x}) > 0$, and every transition preserves the sign invariant. If such a B exists, safe and unsafe regions are provably disjoint. This era contributed the core false-positive elimination engine.

Era III: Python Reality. Making the theory survive Python's execution semantics—bytecode-level control flow, normal and exceptional edges, frame/stack state, dynamic dispatch, unknown library behaviour—required committing to implementation details that killed many elegant claims. Good: they needed to die.

2.3 Inverting the Loop: The Five-Phase Iteration

The AI-slop feedback loop, left to its own devices, is degenerative: each iteration adds noise. The key insight behind A3 is that the same generative breadth that causes the vicious cycle can power a *virtuous* one—provided every AI-generated hypothesis is forced through a gauntlet of formal and empirical checks before it reaches production. What emerged was a five-phase iteration that was not planned but was repeated often enough to become the project's real contribution:

(1) **AI Theorising.** An LLM generates broad hypotheses: new abstractions, candidate proof templates, cross-domain analogies, aggressive architectural combinations.
(2) **Coding.** Ideas are encoded in analysers: bytecode/CFG extraction, symbolic state propagation, unsafe-region checks, barrier template synthesis, dynamic symbolic/concolic validation.
(3) **Testing.** The expensive truth step: synthetic suites, regression tests, large-repo scans, confidence calibration, triage audits.
(4) **Fixing Code.** Typical breakages: path explosion, over-conservative unknown-call handling, context loss across call boundaries, duplicate floods, false positives on guard-heavy code.
(5) **Fixing Theory.** The underappreciated step: instead of forcing code to match a brittle theory, the theory itself is patched—definitions tightened, assumptions made explicit, proof obligations split by semantics layer.

The loop then restarts—but unlike the unchecked slop loop, each iteration *improves* the system rather than degrading it. This is the inverted loop in practice: generate aggressively, then subject everything to adversarial execution. The asymmetry is what makes the cycle virtuous rather than vicious: upstream AI *expands* the hypothesis space; midstream formal/static machinery *prunes* it brutally; downstream agentic AI handles the hard residue. Because the pruning is deterministic and auditable, noise is destroyed rather than propagated, and the surviving signal feeds back into better hypotheses in the next round.

3 The A3 Analysis Engine

3.1 Architecture Overview

A3's pipeline has seven stages:

(1) **Call-graph construction** from Python bytecode.
(2) **Crash-summary computation**: per-function summaries of operations that can raise (division, attribute access, indexing, assertions).
(3) **Guard detection**: identification of explicit checks (`if x is not None`, `isinstance`, `try/except`) that render a potential crash unreachable.
(4) **Barrier-certificate proofs**: 8 barrier patterns that compositionally prove false positives (assume-guarantee contracts, refinement types, inductive invariants, factory post-conditions, disjunctive barriers, control-flow barriers, dataflow barriers, callee return-guarantee safety).
(5) **Kitchensink portfolio verification**: orchestration of 18 SOTA techniques for the surviving candidates.

(6) **Z3-backed DSE confirmation**: directed symbolic execution to confirm or refute surviving findings.
(7) **Deduplication and scoring**: confidence-weighted ranking and SARIF output.

3.2 Bug Taxonomy

A3 implements detectors for 67 bug types organised in four families:

- **Crash bugs (19 types):** `NULL_PTR`, `DIV_ZERO`, `BOUNDS`, `TYPE_CONFUSION`, `ASSERT_FAIL`, `STACK_OVERFLOW`, `USE_AFTER_FREE`, `UNINIT_MEMORY`, `DOUBLE_FREE`, `INTEGER_OVERFLOW`, `FP_DOMAIN`, `MEMORY_LEAK`, `NON_TERMINATION`, `ITERATOR_INVALID`, `DATA_RACE`, `DEADLOCK`, `SEND_SYNC`, `INFO_LEAK`, `TIMING_CHANNEL`.
- **Security bugs (39 types):** Injection (`SQL_INJECTION`, `COMMAND_INJECTION`, `CODE_INJECTION`, `PATH_INJECTION`, ...), web (`REFLECTED_XSS`, `SSRF`, `CSRF_PROTECTION_DISABLED`), serialisation (`UNSAFE_DESERIALIZATION`, `XXE`), cryptography (`WEAK_CRYPTO_KEY`, `BROKEN_CRYPTO_ALGORITHM`), filesystem and network.
- **Exception bugs (fine-grained):** `VALUE_ERROR`, `RUNTIME_ERROR`, `FILE_NOT_FOUND`, `PERMISSION_ERROR`, `OS_ERROR`, `IO_ERROR`, `IMPORT_ERROR`, etc., each mapped to optimal verification strategies from the kitchensink portfolio.
- **Semantic bugs (24 types):** Contract violations (`PRECONDITION_VIOLATION`, `LISKOV_VIOLATION`), temporal properties (`USE_BEFORE_INIT`, `USE_AFTER_CLOSE`), data flow (`UNVALIDATED_INPUT`, `UNCHECKED_RETURN`).

3.3 The Kitchensink Portfolio

A single verification technique is rarely sufficient across all 67 bug types. A3's *kitchensink* engine (enabled by default; disable with `–no-kitchensink`) treats published verification methods as interoperable components in a verification control loop:

(1) Classify the bug's semantic shape.
(2) Route to the strongest low-cost method first.
(3) Escalate only when proof/counterexample remains unresolved.
(4) Keep competing methods as cross-checks, not decorations.

The portfolio draws from:

- Barrier-certificate foundations for safety separation [2, 3].
- Algebraic proof machinery: Positivstellensatz, SOS/SDP, hierarchy lifting, sparsity, and DSOS/SDSOS speed layers [4–8].
- Property-directed and CHC-style reachability: IC3/PDR [9], Spacer [10], interpolation-based model checking [11].
- Abstraction-refinement: classic CEGAR [12], predicate abstraction [13], lazy interpolation [14].
- Learning/synthesis: ICE [15], Houdini [16], SyGuS [17].
- Compositional reasoning: assume-guarantee [18].

3.4 Multi-Barrier Composition

The core false-positive elimination theorem is a disjunctive composition of 8 barrier patterns:

$$B_{\text{total}} = B_1 \vee B_2 \vee \cdots \vee B_8$$

If *any* barrier B_i holds for a candidate finding, that finding is proven to be a false positive. The 8 patterns are: B_1: assume-guarantee contracts; B_2: refinement types; B_3: inductive invariants; B_4: factory post-conditions; B_5: disjunctive barriers; B_6: control-flow barriers; B_7: dataflow barriers; B_8: callee return-guarantee safety.

On DeepSpeed, this composition eliminates 2 513 of 2 638 raw findings (95.3%), with 100% of interprocedural null-pointer warnings proven false positive.

4 Self-Improvement via BugsInPy

4.1 Background

BugsInPy [1] is a curated database of 493 real bugs extracted from 17 popular open-source Python projects: Ansible, Black, Cookiecutter, FastAPI, HTTPie, Keras, Luigi, Matplotlib, Pandas, PySnooper, Sanic, Scrapy, SpaCy, thefuck, Tornado, tqdm, and youtube-dl. Each bug consists of a buggy commit, a fixed commit, and a minimal test case that exposes the bug.

4.2 Scope Classification

Not every BugsInPy bug is in scope for a static analyser. A3 targets crash bugs (missing null checks, division by zero, out-of-bounds access), security vulnerabilities (injection, deserialisation, path traversal), and exception-related semantic bugs (unguarded `AttributeError`, `TypeError`, `KeyError`, `IndexError`, `ValueError`). Out-of-scope categories include pure logic/algorithmic bugs (wrong formula, wrong constant), formatting/string-output bugs, performance bugs, API-design changes, and build/packaging reorganisations.

We developed a heuristic classifier (`classify_bugsinpy_scope.py`) that analyses each bug's patch to determine whether it falls within A3's detection capabilities, using AST analysis and keyword heuristics over the diff.

4.3 Iterative Self-Improvement Protocol

The self-improvement loop operates as follows:

(1) **Extract.** For each in-scope BugsInPy bug, extract the changed Python file(s) from the patch.
(2) **Scan.** Run A3 on both the buggy and fixed versions of each changed file.
(3) **Classify.** Compare results against ground truth:
    - *True positive*: A3 reports a bug in the buggy version and reports clean in the fixed version.
    - *False negative*: A3 misses the bug (buggy version scans clean).
    - *False positive*: A3 flags the fixed code.
(4) **Improve.** For every false negative or false positive, construct a structured prompt describing the gap and invoke an LLM coding assistant (GitHub Copilot CLI) to patch A3's implementation.
(5) **Verify.** Re-scan the bug to confirm the patch resolves the classification error before proceeding.

This protocol is automated end-to-end (`iterative_bugsinpy_improve.py`). The outer loop repeats indefinitely (interrupted by Ctrl-C); the inner loop iterates over BugsInPy bugs, and on the first false positive or false negative, invokes the LLM to patch A3, then re-scans to verify the fix. State is checkpointed to JSON so the process can be resumed across sessions.

The key insight is that *prior git changes to the analyser*—accumulated over hundreds of iterations of the five-phase loop—serve as training

Table 1: Synthetic benchmark results.

Suite	TP	FP	FN	TN	Prec.	Recall
Standalone	29	2	0	2	93.5%	100%
Multi-file	8	2	0	0	80.0%	100%
Combined	**37**	**4**	**0**	**2**	**90.2%**	**100%**

Table 2: DeepSpeed false-positive elimination pipeline.

Stage	Findings remaining
Raw candidates	4 261
After guard detection (Papers #1–20)	342
After barrier certificates (B_1–B_8)	125
After DSE (Z3)	∼10

signal for future improvements. Each patch to A3 narrows the gap between the analyser's model of Python semantics and the actual runtime behaviour exposed by BugsInPy's real-world bugs.

5 Evaluation

5.1 Synthetic Benchmark

We constructed a synthetic benchmark of 43 programs spanning 67 bug types, including both standalone single-file programs and multi-file projects with interprocedural bug flows. Each program has a known ground-truth label (BUG or SAFE) and a documented bug type.

As shown in Table 1, A3 achieves 90.2% precision at perfect recall, yielding an F_1 score of 0.949. The 4 false positives stem from conservative handling of opaque library calls; the analyser reports a potential bug rather than assuming safety for unknown code.

5.2 DeepSpeed Case Study

We ran A3 on Microsoft DeepSpeed, a widely-used deep learning optimisation library with approximately 150 k lines of Python across 7 826 functions.

Of the 4 261 raw candidates, 95.7% (4 077) were proven false positives by guard detection and barrier certificates. Of the surviving 184 candidates subjected to deep manual investigation, we confirmed:

- **6 genuine bugs**: unguarded division-by-zero reachable from user-supplied configuration values.
- **4 borderline bugs**: division-by-zero reachable only from malformed configuration.
- **174 false positives**: attributed to framework guarantees (`self` never `None`), pytest fixtures, argparse defaults, and HuggingFace config invariants.

Example: `_ensure_divisibility`. In `deepspeed/utils/groups.py`, the function:

```
def _ensure_divisibility(numerator,
    denominator):
    assert numerator % denominator == 0, \
        '{} is not divisible by {}'.format(
            numerator, denominator)
```

is called 7 times with user-configurable parallel sizes. If a user passes `model_parallel_size=0` in their DeepSpeed JSON configuration, `numerator % denominator` raises `ZeroDivisionError` *before* the assertion message fires.

Example: `ThroughputTimer`. In `deepspeed/utils/timer.py`:

```
def _is_report_boundary(self):
    if self.steps_per_output is None:
        return False
    return self.global_step_count \
           % self.steps_per_output == 0
```

The `None` check passes when `steps_per_output=0`, but the subsequent modulo crashes.

Example: Silent corruption in FP quantisation. In `op_builder/hpu/fp_quantize`

```
dequant_out = torch.ops.hpu.cast_from_fp8(
    input_q, (1.0 / scale), orig_dtype)
```

If any quantisation group has all-zero values, `scale` is zero, producing `inf` or `nan` that *silently* corrupts model weights—no crash, no error message, just wrong inference results.

5.3 Microsoft Repository Portfolio

We evaluated A3 across 15 Microsoft-maintained open-source repositories and confirmed true-positive findings in several:

- **Azure Counterfit**: 4 instances of `eval()` on unsanitised CLI input enabling arbitrary code execution (80% exploitability).
- **RD-Agent**: Tarfile path traversal (`tar.extractall` without validation on Kaggle downloads) and `eval()` on generated code templates.
- **RESTler**: `shell=True` subprocess calls with path-derived arguments in build and quickstart scripts.
- **FLAML**: Division-by-zero in feature normalisation (`prep["scale"]` can be zero for constant features) and empty regret-matrix handling.
- **Presidio**: Regex injection / ReDoS risk in custom PII recogniser patterns.
- **DeepSpeed**: Pickle deserialisation of untrusted checkpoint files (`ds_model_config.pkl`).

5.4 Comparison with CodeQL

A3 implements a mapping from CodeQL's Python security query suite to its own detectors, covering injection, deserialisation, XSS, SSRF, path traversal, cryptographic, and configuration bug classes. The security detector registry covers all 39 CodeQL-equivalent security bug types plus 28 additional crash, exception, and semantic bug types that CodeQL's Python queries do not address.

6 Agentic Triage

Findings that survive the static pipeline—typically around 1% of raw candidates—are forwarded to an *agentic LLM triage* stage. Rather than asking an LLM to classify a finding from a one-shot prompt, A3 equips the LLM with tools to *explore* the codebase:

- `read_file`: read specific source ranges.
- `search_codebase`: grep for guard patterns, callers, imports.
- `get_function_source`: retrieve the full function body.
- `list_directory`: explore project structure.

The LLM agent conducts a multi-turn investigation—reading the flagged function, checking callers, inspecting tests, following imports—before producing a TP/FP classification with a rationale. This agentic approach replaces the fragile one-shot classification that characterises most LLM-triage systems.

In CI (GitHub Actions), the triage uses the `GITHUB_TOKEN` already available on every runner to access GitHub Models, requiring zero additional API key configuration.

7 CI Integration and Baseline Ratcheting

A practical design choice that makes A3 deployable in large existing codebases is *baseline ratcheting*:

- Existing accepted findings are recorded in `.a3-baseline.json`.
- New unaccepted findings fail CI.
- Disappearing findings are auto-pruned from the baseline.

This shifts the developer experience from "infinite backlog" to "no net new risk," which is the only sustainable adoption model for large existing codebases. Adding A3 to any repository takes 60 seconds:

```
pip install a3-python[ci]
a3 init . --copilot
git add .github/ .a3.yml .a3-baseline.json
git commit -m "ci: add a3 static analysis"
git push
```

8 Related Work

Python Static Analysis. Pyright [?], mypy [?], and Pytype [?] focus on type checking rather than bug detection. Bandit [?] applies pattern-matching for security issues but lacks semantic analysis. CodeQL [?] provides a powerful dataflow framework but requires QL expertise and does not target crash bugs. A3 complements these tools by combining crash, security, and semantic bug detection with formal false-positive elimination.

Barrier Certificates. Prajna et al. [2, 3] introduced barrier certificates for hybrid system safety verification. We adapt the core idea—constructing a separating function between safe and unsafe regions—to the discrete, exception-rich domain of Python program states.

LLM-Assisted Analysis. Recent work has explored using LLMs to generate static-analysis rules [?], triage findings [?], and repair bugs [?]. A3 differs in placing LLM judgment *after* deterministic static analysis, not instead of it.

BugsInPy. Widyasari et al. [1] curated BugsInPy as a benchmark for testing and debugging research. We use it not only for evaluation but as a *self-improvement driver*: classification errors on BugsInPy bugs trigger automated patches to the analyser.

9 Threats to Validity

Internal. The synthetic benchmark was constructed by the authors and may not represent the full diversity of real-world bugs. We mitigate this with the BugsInPy and DeepSpeed evaluations.

External. Results on DeepSpeed and 15 Microsoft repositories may not generalise to all Python codebases, particularly those with heavy metaprogramming or C extensions.

Construct. The distinction between "true positive" and "false positive" for division-by-zero bugs depends on whether user-configurable values can realistically be zero. We report borderline cases separately.

10 Conclusion

The AI-slop feedback loop—in its original, unchecked form—is a vicious cycle that degrades every tool it touches. But it is not inevitable. By placing deterministic static analysis—barrier certificates, Z3-backed symbolic execution, and a portfolio of 18 verification techniques—*before* LLM judgment, A3 inverts the loop: each iteration destroys noise rather than propagating it, and the surviving signal feeds back into better hypotheses, turning a historically negative phenomenon into a virtuous self-improvement cycle.

On synthetic benchmarks, A3 achieves $F_1 = 0.949$. On DeepSpeed, the barrier-certificate engine alone eliminates 95.3% of false positives, leaving confirmed bugs that include division-by-zero vulnerabilities reachable from user configuration and a silent data-corruption issue in FP quantisation. The BugsInPy self-improvement protocol demonstrates that real-world bug databases can drive automated analyser patches, closing the gap between the analyser's model and Python's runtime semantics.

A3 is available at `pip install a3-python` under an open-source licence.

References

[1] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tran, H. Hata, S. Chandrasekaran, C. Yang, and D. Lo. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proc. ESEC/FSE*, 2020.
[2] S. Prajna, A. Jadbabaie, and G. J. Pappas. Safety verification of hybrid systems using barrier certificates. In *Proc. HSCC*, 2004.
[3] S. Prajna, A. Jadbabaie, and G. J. Pappas. A framework for worst-case and stochastic safety verification using barrier certificates. *IEEE Trans. Automatic Control*, 52(8):1415–1428, 2007.
[4] M. Putinar. Positive polynomials on compact semi-algebraic sets. *Indiana Univ. Math. J.*, 42(3):969–984, 1993.
[5] P. A. Parrilo. Semidefinite programming relaxations for semialgebraic problems. *Math. Programming*, 96(2):293–320, 2003.
[6] J.-B. Lasserre. Global optimization with polynomials and the problem of moments. *SIAM J. Optim.*, 11(3):796–817, 2001.
[7] M. Kojima, S. Kim, and H. Waki. Sparsity in sums of squares of polynomials. *Math. Programming*, 103(1):45–62, 2005.
[8] A. A. Ahmadi and A. Majumdar. DSOS and SDSOS optimization: more tractable alternatives to sum of squares and semidefinite optimization. *SIAM J. Appl. Algebra Geometry*, 3(2):193–230, 2019.
[9] A. R. Bradley. SAT-based model checking without unrolling. In *Proc. VMCAI*, 2011.
[10] A. Komuravelli, A. Gurfinkel, and S. Chaki. SMT-based model checking for recursive programs. In *Proc. CAV*, 2014.
[11] K. L. McMillan. Interpolation and SAT-based model checking. In *Proc. CAV*, 2003.
[12] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *Proc. CAV*, 2000.
[13] E. Clarke, D. Kroening, N. Sharygina, and K. Yorav. Predicate abstraction of ANSI-C programs using SAT. *Formal Methods in System Design*, 25(2–3):105–127, 2004.
[14] K. L. McMillan. Lazy abstraction with interpolants. In *Proc. CAV*, 2006.

[15] P. Garg, C. Löding, P. Madhusudan, and D. Neider. ICE: A robust framework for learning invariants. In *Proc. CAV*, 2014.
[16] C. Flanagan and K. R. M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. FME*, 2001.
[17] R. Alur, R. Bodik, G. Juniwal, M. M. K. Martin, M. Raghothaman, S. A. Seshia, R. Singh, A. Solar-Lezama, E. Torlak, and A. Udupa. Syntax-guided synthesis. In *Proc. FMCAD*, 2013.
[18] T. A. Henzinger, S. Qadeer, and S. K. Rajamani. You assume, we guarantee: methodology and case studies. In *Proc. CAV*, 1998.